



# Web services

---

Prof. Alessandro Brawerman

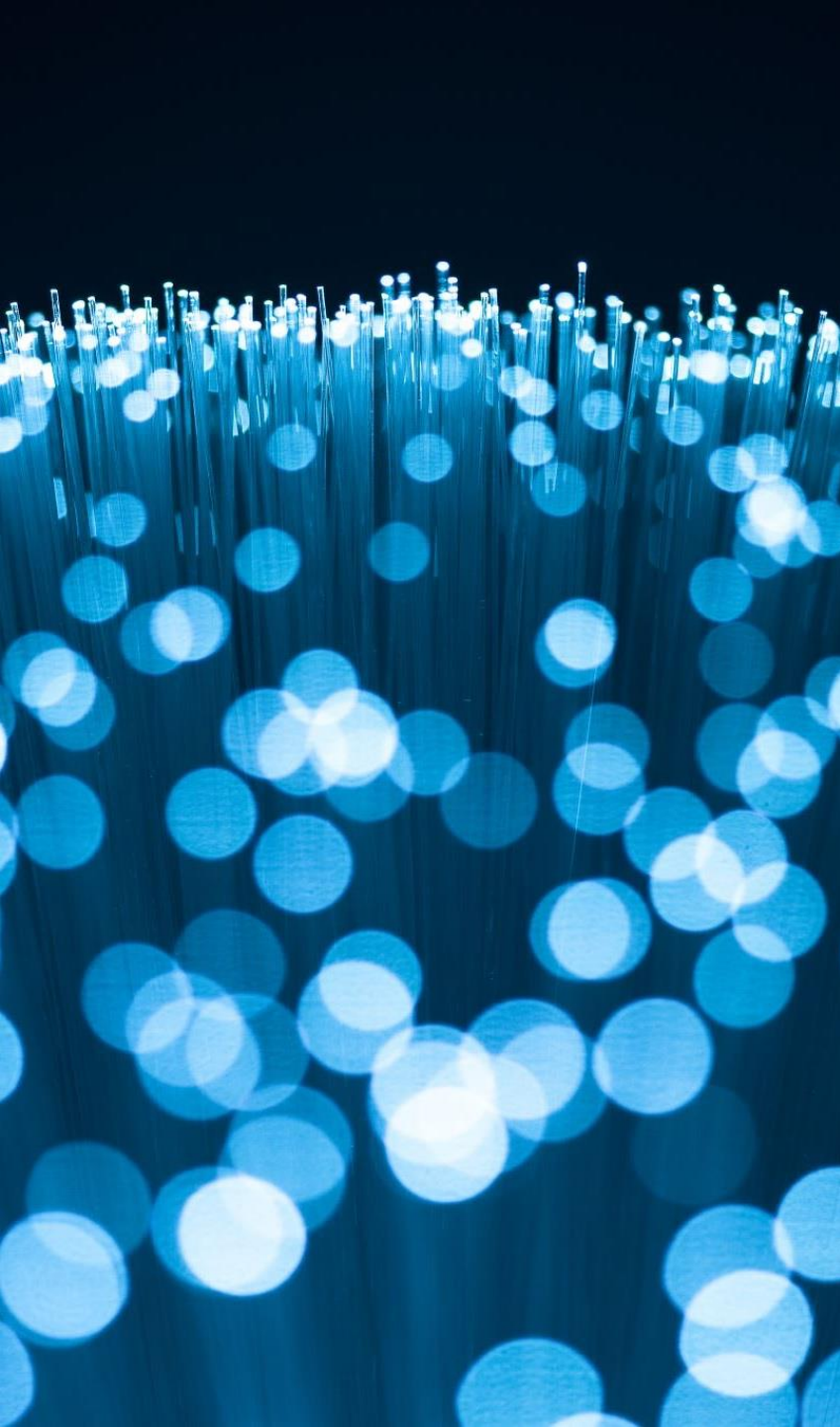


---

# Web service

- Um Web Service é uma aplicação ou componente que comunica dados via algum protocolo de transferência de dados, sendo o mais comum o HTTP.
- Exemplos:
  - APIs de clima
  - APIs de câmbio de moedas
  - APIs proprietários
- Tipos de Web Services
  - REST (mais popular, utiliza JSON para troca de dados)
  - SOAP (mais antigo, usa XML).





# Web service

- A combinação mais comum no mercado, em especial para Mobile, é:
  - REST + HTTP + JSON
- Estes serviços são chamados de Restfull
- Combinam a arquitetura REST com HTTP para comunicação de dados.
- Além disto o JSON é usado como parser.
- Esta combinação é mais leve e menos burocrática que o uso de SOAP + XML.



# Retrofit2

A type-safe HTTP Client for Android and Java



The image features a green background on the left side, which contains the Retrofit logo. The logo consists of the word "retrofit" in a large, white, lowercase, sans-serif font. Below it, in a smaller, white, uppercase, sans-serif font, is the text "HTTP Client for Android". A white hand icon is pointing towards the text. A white arrow points from the left edge of the image towards the hand.

# retrofit

HTTP Client for Android

## Retrofit

- Biblioteca de alto nível que é construída sobre OkHttp e facilita o trabalho com APIs REST.
- Ela oferece uma forma mais simples de lidar com requisições, respostas e conversões de dados (por exemplo, JSON para objetos Kotlin).
  - Torna a API de HTTP em uma Interface Java
  - Usa anotações (tags) para descrever os requests

# Retrofit



Mais simples e estruturada

Simplifica a criação de endpoints de API e o tratamento de respostas.



Suporte a parsers

Facilmente integra conversores para transformar a resposta JSON em objetos Kotlin.



Suporte a corrotinas

Tem integração nativa com corrotinas, facilitando o uso de funções de suspensão para chamadas assíncronas.

# Retrofit – Como usar?

---

- É necessário baixar a biblioteca Retrofit para seu projeto.
- Adicionar as dependências do Retrofit.
- No build.gradle(app) insira a dependência

```
implementation("com.squareup.retrofit2:retrofit:2.9.0")
```

```
implementation("com.squareup.retrofit2:converter-gson:2.9.0")
```

- No Android Manifesto insira o pedido de permissão para Internet, antes do bloco <application>

```
<!-- Permissão para acessar a Internet -->
```

```
<uses-permission android:name="android.permission.INTERNET" />
```

# Retrofit – Como usar?

---

- Criar a interface da API
- A interface define os endpoints da API e os métodos HTTP que você deseja usar.

```
// Define a interface da API
interface BlockchainApi {
    // Define um endpoint GET que retorna o ticker do Bitcoin
    @GET("ticker")
    suspend fun getBitcoinTicker(): Map<String, Any>
}
```

- A função `getBitcoinTicker()` faz uma chamada GET para o endpoint `/ticker`.
- O uso de `suspend` indica que a função será executada dentro de uma corrotina.



# Retrofit – Como usar?

---

- Configurar o Retrofit e fazer a requisição a partir de corrotinas.

```
// Configurando o Retrofit
val retrofit = Retrofit.Builder()
    .baseUrl("https://blockchain.info/")
    .addConverterFactory(GsonConverterFactory.create())
    .build()

// Cria uma implementação da interface BlockchainApi
blockchainApi = retrofit.create(BlockchainApi::class.java)
```



# CatFact APP

- O app faz uma requisição para uma API e exibe o resultado em um TextView.
- Usaremos a Cat Facts API para buscar fatos aleatórios sobre gatos.
- URL da API:  
<https://catfact.ninja/fact>

# CatFact APP

---

- Crie um novo projeto no Android Studio.
- Adicione as dependências do Retrofit no build.gradle do app.
- Adicione a permissão de Internet no arquivo manifesto.
- Crie um novo arquivo Kotlin/Class e escolha a opção Data class – chame de CatFact

```
data class CatFact(  
    val fact: String,  
    val length: Int  
)
```

# CatFact APP

---

- Agora vamos acertar a interface para a API
- Crie um novo arquivo Kotlin/Class e escolha a opção Interface – chame de CatFactApi

```
interface CatFactApi {  
    @GET("fact")  
    suspend fun getCatFact(): CatFact  
}
```

## CatFact APP - View

```
<LinearLayout
```

```
xmlns:android="http://schemas.android.com/apk/res/android"
```

```
    android:layout_width="match_parent"
```

```
    android:layout_height="match_parent"
```

```
    android:orientation="vertical"
```

```
    android:padding="16dp">
```

```
    <TextView
```

```
        android:id="@+id/tvResult"
```

```
        android:layout_width="match_parent"
```

```
        android:layout_height="wrap_content"
```

```
        android:text="Clique para obter um fato sobre  
gatos"
```

```
        android:textSize="18sp"
```

```
        android:paddingTop="30dp"
```

```
        android:layout_weight="1"/>
```



## CatFact APP - View

```
<!-- ProgressBar invisível inicialmente -->
<ProgressBar
    android:id="@+id/progressBar"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:visibility="gone" />

<Button
    android:id="@+id/btnGetFact"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Get Fact"
    android:onClick="getFact"/>

<View
    android:layout_width="0dp"
    android:layout_height="40dp" />

</LinearLayout>
```

# CatFact APP Controller

- Para finalizar, vamos ao Controller principal.
- Inicie com a declaração de objetos globais e inicialização no onCreate.

```
private lateinit var tvResult: TextView
private lateinit var progressBar: ProgressBar
private lateinit var catFactApi: CatFactApi

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    enableEdgeToEdge()
    setContentView(R.layout.activity_main)

    tvResult = findViewById(R.id.tvResult)
    progressBar = findViewById(R.id.progressBar)
}
```

# CatFact APP - Controller

---

- Ainda no onCreate, configure o retrofit para ser usado a partir da URL base.

```
// Configura o Retrofit
val retrofit = Retrofit.Builder()
    .baseUrl("https://catfact.ninja/")
    .addConverterFactory(GsonConverterFactory.create())
    .build()

    catFactApi = retrofit.create(CatFactApi::class.java)
} //Fim onCreate()
```

# CatFact APP - Controller

- Ao clicar no botão Get Fact o método `getFact` é chamado para fazer o request.
- Note que ele inicia uma corrotina que irá durar durante o ciclo de vida desta activity.
- O dispatcher é modificado de Main para IO para realizarmos o request.
- Ao final do request voltamos ao dispatcher Main para atualização da View com a resposta obtida.

# CatFact APP - Controller

```
fun getFact(view: View) {  
    // Mostra progress bar  
    progressBar.visibility = View.VISIBLE  
    tvResult.visibility = View.GONE  
  
    // Inicia uma coroutine para fazer a requisição em segundo plano  
    lifecycleScope.launch {  
        try {  
            // Faz a requisição em segundo plano  
            val factResponse = withContext(Dispatchers.IO) {  
                catFactApi.getCatFact()  
            }  
        }  
    }  
}
```

# CatFact APP - Controller

```
// Mata progress bar
progressBar.visibility = View.GONE
tvResult.visibility = View.VISIBLE
// Atualiza o TextView com o resultado
tvResult.text = factResponse.fact

} catch (e: Exception) {
    Log.e(tag: "MainActivity", msg: "Erro ao buscar o fato", e)
    tvResult.text = "Erro ao buscar o fato."
}

} //Fim da corrotina
} //Fim getFact
} //Fim da activity
```



# CatFact APP – Execute o APP

